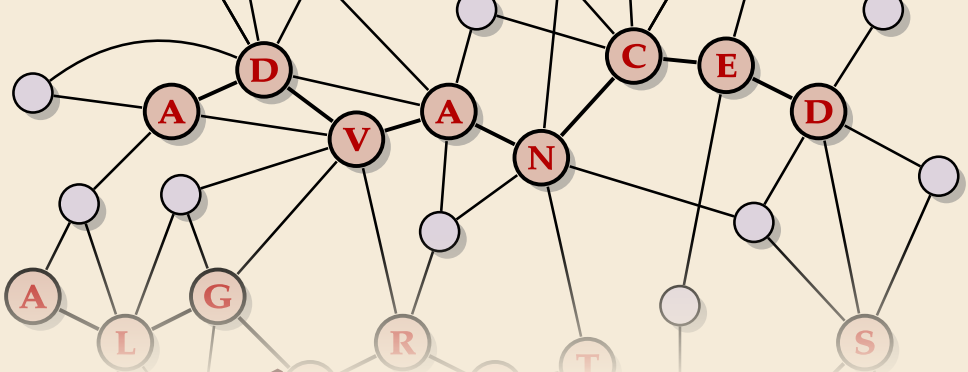


1 Motivation & Outline

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-10-12 14:47`

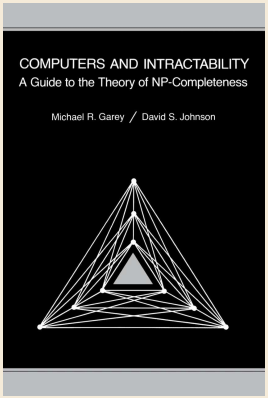
1 Motivation & Outline

1.1 How to Solve Hard Problems?

1.1 How to Solve Hard Problems?

View on NP-completeness in introductory courses





"I can't find an efficient algorithm, but neither can all these famous people."

Garey, Johnson 1979

. . . but this is not the end of the story!

"you had just neatly sidestepped potential charges of incompetence by proving that the bandersnatch problem is NP-complete. However, the bandersnatch problem had ***refused to vanish at the sound of those mighty words****, and you were still faced with the task of finding some usable algorithm for dealing with it."*

Garey, Johnson 1979, Chapter 6

. . . but this is not the end of the story!

"you had just neatly sidestepped potential charges of incompetence by proving that the bandersnatch problem is NP-complete. However, the bandersnatch problem had ***refused to vanish at the sound of those mighty words****, and you were still faced with the task of finding some usable algorithm for dealing with it."*

Garey, Johnson 1979, Chapter 6

⇝ Look for "loopholes" in the impossibility results:

- hard only for large integers in input? ⇝ *pseudopolynomial* algorithms
- hard only when some parameter is large? ⇝ *fixed-parameter* algorithms
- hard only for exact solution? ⇝ *approximation* algorithms
- hard only for unlucky choices of the algorithm? ⇝ *randomized* algorithms
- hard only on contrived inputs? ⇝ *average-case* / *smoothed* analysis (only in passing here)
- hard only without a quantum computer? ⇝ *quantum* algorithms (not here)
- . . .

also quantum computers are not here

Algorithms for hard problems

- for each possible loophole left by NP-hardness, we consider
  - algorithmic techniques that exploit the loophole
  - impossibility results showing when that exploit fails to apply
  - examples of problems where this loophole does (not) help

Algorithms for hard problems

- for each possible loophole left by NP-hardness, we consider
  - algorithmic techniques that exploit the loophole
  - impossibility results showing when that exploit fails to apply
  - examples of problems where this loophole does (not) help

- focus is on provable results & eternal truths
  theoretical algorithms and complexity theory
  - *no algorithm without a guarantee*
  - ultimate goal is a practically relevant solution,
    but will often strive for simpler, "pedagogical" examples here
  - strong foundation in math and theoretical computer science needed

Loophole 1: Large integers

- **Loophole:** problem only hard when the integers that are part of the input are huge
- **Algorithmic idea:** ***Pseudopolynomial algorithms***
  running time polynomial in input size **and** largest integer *value*

⇝ can be efficient if only small-integer inputs needed

- **Impossibility results:** strong NP-hardness

Loophole 2: Small parameters

- **Loophole:** problem only hard when some *parameter* k of the input is large
  (size of the sought solution, number of variables, treewidth, . . .)
- **Algorithmic idea:** ***Fixed-parameter algorithms***
  running time $f(k) \cdot \mathrm{poly}(n)$ for input size n:
  combinatorial explosion confined to the *parameter*, polynomial in everything else

$\rightsquigarrow$ can be efficient if only small-parameter inputs needed
($2^k \cdot n^2$ with $k = 20$ is fine; n^{20} is not)

- **Algorithmic idea 2:** ***Efficient exponential algorithms***
  reduce the base of exponential growth to delay combinatorial explosion (a bit)
- **Impossibility results:** $\mathsf{W}[P]$-hardness, $\mathsf{W}[1]$-hardness, lower bounds from ETH

Loophole 3: Unlucky Choices

- **Loophole:** an algorithm's bad cases are triggered by specific "unlucky" decisions (choice of pivot, order of exploration, . . .) that an adversarial input can provoke
- **Algorithmic idea:** ***Randomized algorithms***
  algorithm can decide randomly
  $\rightsquigarrow$ *no fixed input* is bad all of the time;
  accept a small probability of a wrong answer (*Monte Carlo*)
  or of a slow run (*Las Vegas*)
- **Related:** *Average-case* and *smoothed analysis* randomize the *input* instead
  (input-side counterpart of this loophole; not our focus, but see simplex in Unit 6)
- **Impossibility results:** randomized complexity classes (ZPP, RP, BPP);
  derandomization: under plausible hardness assumptions
  BPP = P,
  so randomness alone does not escape NP-hardness

Loophole 4: Exact answers

- **Loophole:** optimization problem only hard when precisely optimal solution requested
- **Algorithmic idea:** ***Approximation algorithms***, PTAS, FPTAS
  polytime through compromise on quality
  but with a provable guarantee on how much worse results get!
- **Impossibility results:** hardness of approximation, APX-hardness

Overview of the module

Goals:

- enhance your toolbox of algorithmic methods and techniques
  - ⇝ here: focus on hard problems
- enhance your toolbox of hardness and lower-bounding techniques
- focus on algorithm theory and rigorous methods
  - ⇝ formal proofs, mathematical machine models, asymptotic analysis

Overview of the module

Goals:

- enhance your toolbox of algorithmic methods and techniques
  - ⇝ here: focus on hard problems
- enhance your toolbox of hardness and lower-bounding techniques
- focus on algorithm theory and rigorous methods
  - ⇝ formal proofs, mathematical machine models, asymptotic analysis

Units: (preliminary plan)

	0. Administrativa		
	1. Motivation & Outline	*Loophole 3*	*7.* Randomization Basics
Foundations	*2.* Complexity Theory Recap		*8.* Randomized Complexity
Loophole 1	*3.* Pseudopolynomial Algorithms		*9.* Random Tricks
Loophole 2	*4.* Fixed-Parameter Algorithms	*Loophole 4*	*10.* Approximation Algorithms
	5. Parameterized Hardness		*11.* LP-Based Approximation
	6. Advanced Parameterized Ideas		*12.* Inapproximability